Kinematic Modelling and MATLAB-Based Analysis of the Dobot Magician Lite Robot Arm

1 Introduction

Robot manipulators are widely used across industrial, medical and educational domains because of their precision, repeatability and programmability [6]. Common applications include pick-and-place operations, assembly lines, welding, and laboratory automation. In educational and research settings, compact systems such as the Dobot Magician Lite are used for teaching robotics concepts and for prototyping control algorithms before deployment.

The Dobot Magician Lite is a desktop-scale robot manipulator consisting of four revolute joints (J1–J4), forming an articulated robotic arm structure [8]. It operates in both point and Cartesian coordinate systems, enabling flexible motion control and trajectory planning. The robot supports multiple motion modes (point to point, point-to-point (PTP) linear (MOVL), and arc-based trajectories, making it suitable for both academic and practical applications [8].

Accurate kinematic modelling is essential for predicting end-effector pose and for implementing trajectory tracking and inverse kinematics. Compared to parallel manipulators, serial manipulators offer a larger workspace and greater flexibility, but suffer from lower stiffness and accumulated positioning errors. Compared to SCARA robots, the Dobot provides greater vertical flexibility but may have reduced speed and precision in planar tasks.

The Dobot Magician Lite is particularly suitable for educational use because its open-source ecosystem and documentation support rapid prototyping. The manufacturer lists a rated payload capacity (250 g), limited reach (340 mm), and lower industrial-grade precision compared to high-end robotic systems [8]. In this case of this project we was to develop an analytical and MATLAB-based model of the Dobot Magician Lite and to evaluate its forward kinematics, inverse kinematics, Jacobian, singularities and validation behaviour.

List of Symbols

- θ_1 – base rotation angle
- θ_2 – shoulder joint angle
- θ_3 – elbow joint angle
- θ_4 – end-effector rotation angle
- L_1, L_2, L_3 – main arm link lengths

2 Kinematic Modelling

2.1 Robot configuration and assumptions

The kinematic model therefore treats the robot as a 4R serial manipulator, with three joints controlling the position and one joint controlling the orientation of the end-effector.

Although the physical robot includes a parallelogram mechanism [4] that helps maintain end-effector orientation, the kinematic model used in this work represents that behaviour through an equivalent passive transformation ($-\theta_3 - \theta_2$), so the parallelogram is not treated as an independent kinematic degree of freedom, but only as a modelling device used to preserve the geometric constraint.

Figure 1: Appearance of the Dobot Magician Lite showing the physical arrangement of the four revolute joints used in the kinematic model. Adapted from [8].

2.2 Denavit–Hartenberg parameterisation

Figure 2: Joint coordinate system of the Dobot Magician Lite showing the four revolute joints considered in the kinematic model. Adapted from [8].

Figure 3: Denavit–Hartenberg frame assignment used for the kinematic model of the Dobot Magician Lite manipulator.

The Denavit–Hartenberg parameters table derived from the frame assignment is given below.

Table 1: Denavit–Hartenberg parameters for the Dobot Magician Lite manipulator.

Link	a_i (mm)	α_i (rad)	d_i (mm)	θ_i
1 - Base	0	$\frac{\pi}{2}$	138	θ_1
2 - Rear Arm	150	0	0	θ_2
3 - Fore Arm	150	0	0	θ_3
P - unconstrictable link	0	$\frac{\pi}{2}$	90	$-\theta_3 - \theta_2$
4 - End-Effector	0	0	0	θ_4

The coordinate frames were assigned according to the standard Denavit-Hartenberg convention [9]. Each joint was aligned with the axis of motion of the corresponding joint, while each x-axis was defined along the common normal between adjacent axes. The physical robot has four revolute joints, but, however, an additional passive transformation was introduced only to capture the parallelogram mechanism that keeps the end-effector orientation approximately constant. This passive term is not an independent joint and does not increase the robot's actuated degrees of freedom.

The Denavit-Hartenberg convention provides a systematic framework for assigning link parameters and deriving homogeneous transformations between adjacent coordinate frames [2, 9].

2.3 Forward Kinematics

The forward kinematics of the Dobot Magician Lite were derived using the Denavit-Hartenberg convention. A homogeneous transformation matrix was assigned to each joint in order to describe the position and orientation of one frame relative to the previous frame. The overall forward kinematics of the base frame to the end-effector frame was then obtained by multiplying these individual transformations [2, 9].

For the equivalent kinematic model used in this project, the total transformation is written as

$$T_{04} = A_1 A_2 A_3 A_4$$

where A_i denotes the passive transformation introduced to represent the parallelogram constraint in the physical robot. The passive term is included only to preserve the geometric behaviour of the end-effector and does not represent an additional actuated degree of freedom.

The final homogeneous transformation matrix provides both the orientation of the end-effector and the Cartesian position of the end-effector. In this case, the first terms of this transformation were used to derive the inverse kinematics and the Jacobian matrix, which were then used for velocity and singularity analysis. TODO: This is wrong as we need L1 which is zero.

$$T_{04} = \begin{bmatrix} \cos(\theta_1 + \theta_2) & 0 & \sin(\theta_1 + \theta_2) & \cos(\theta_1)(L_3\cos(\theta_3) + L_2\cos(\theta_2) + L_1) \\ \sin(\theta_1 + \theta_2) & 0 & -\cos(\theta_1 + \theta_2) & \sin(\theta_1)(L_3\cos(\theta_3) + L_2\cos(\theta_2) + L_1) \\ 0 & 1 & 0 & h + L_3\sin(\theta_3) + L_2\sin(\theta_2) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Where: θ_1 is the base rotation, θ_2 is the shoulder joint angle, θ_3 is the elbow joint angle, θ_4 is the wrist rotation, L_1 is the base offset, L_2 is the rear arm length, L_3 is the fore arm length, h is the base offset. Note that L_1 is not included in the Jacobian because it is a constant term that does not affect the velocity mapping.

2.4 Jacobian and singularity analysis

The Jacobian matrix relates joint rates to end-effector motion. In this work, the Jacobian was used to relate joint velocities to end-effector linear and angular velocity and to find singular configurations where the end-effector motion is restricted. The Jacobian was derived by differentiating the forward kinematics in terms of the selected task variables [2]. The Jacobian formed analytically is valid for all kinematically feasible configurations in which the manipulator loses mobility or becomes difficult to control.

Geometric Jacobian

The geometric Jacobian relates the joint velocities to the linear and angular velocity of the end effector:

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = J_g(q)\dot{q}$$

The geometric Jacobian is therefore:

$$J_g(q) = \begin{bmatrix} -\sin(\theta_1)(L_3\cos(\theta_3) - L_2\sin(\theta_2) + \theta_1) & -\cos(\theta_1)(L_3\sin(\theta_3) + L_2\cos(\theta_2)) & -L_3\cos(\theta_1+\theta_3)\cos(\theta_1) \\ \cos(\theta_1)(L_3\cos(\theta_3) - L_2\sin(\theta_2) + \theta_1) & -\sin(\theta_1)(L_3\sin(\theta_3) + L_2\cos(\theta_2)) & -L_3\cos(\theta_1+\theta_3)\sin(\theta_1) \\ 0 & L_3\cos(\theta_3) + L_2\cos(\theta_2) & L_3\sin(\theta_1+\theta_3) \\ 0 & \sin(\theta_1) & \sin(\theta_1+\theta_3) \\ 0 & -\cos(\theta_1) & -\cos(\theta_1+\theta_3) \\ 1 & 0 & 0 \end{bmatrix}$$

The base offset h appears in the forward kinematics of z, but it does not appear in the Jacobian because it is a constant term. Its derivative with respect to all joint variables is zero.

Singularities

A singularity occurs when the determinant of the analytical Jacobian becomes zero:

$$\det J_a(q) = 0 \Rightarrow \det J_g(q) = 0$$

This expression shows that singularities occur when either $\sin(\theta_3) = 0$ or $r = 0$. The condition $\sin(\theta_3) = 0$ corresponds to the arm being fully stretched or fully folded, so that the links become collinear. The condition $r = 0$ corresponds to the wrist center lying on the base axis, so small joint motions can rotate the arm in certain horizontal directions. Near these configurations, the velocity mapping becomes ill-conditioned and large joint velocities may be required to generate small end-effector motion [2].

2.5 Inverse Kinematics

The inverse kinematics problem was solved using a geometric approach, because it provides closed-form solutions and is well suited to real-time control [2].

Figure 4: Top-view geometry used to compute the base joint angle θ_1.

The base joint angle was obtained from the projection of the target position:

$$\theta_1 = \operatorname{atan2}\left(\frac{y}{x}\right)$$

The radial distance in the horizontal plane is then $r = \sqrt{x^2 + y^2}$. The remaining problem is reduced to a planar two-link geometry in the $r - z$ plane [2]. A geometric inverse-kinematics method was selected because it provides closed-form solutions and is well suited to real-time joint-space control [2].

R-Z Plane

Figure 5: Planar geometric construction used for the inverse kinematics solution.

Figure 5 shows the planar geometry used to derive the shoulder and elbow solutions. Two solutions besides exist for reachable targets: an elbow-up configuration and an elbow-down configuration. Accordingly, the shoulder angle may be written as

$$\theta_2 = \alpha \pm \beta$$

for one branch and

$$\theta_2 = \alpha - \beta$$

for the other. The elbow angle θ_3 is then obtained from the corresponding trigonometric relation, giving two valid inverse-kinematics solutions for the same target position.

3 Materials and Methods

3.1 Kinematic Modelling Approach

The kinematic model of the Dobot Magician Lite was developed using the Denavit–Hartenberg convention. Homogeneous transformation matrices were used to describe the spatial relationship between adjacent links. Each joint was modelled as a revolute joint, and the robot was approximated as an open kinematic chain, despite the parallelogram mechanism in the physical mechanism. A passive link was introduced in the model to maintain the constraint that the end-effector remains parallel to the ground. This simplification allows the system to be analysed using standard serial manipulator techniques while preserving key geometric behaviour.

3.2 Inverse Kinematics Strategy

A geometric approach was selected for solving the inverse kinematics problem. This method was chosen due to the relatively simple structure of the robot and the availability of clear geometric relationships between joint angles and the end-effector position.

The inverse kinematics solution was derived by:

- First solving for the base rotation using $\theta_1 = \operatorname{atan2}(y, x)$
- Reducing the 3D problem into a 2D planar problem in the (r, z) plane
- Applying trigonometric relationships and cosine law to determine θ_2 and θ_3
- Considering both elbow-up and elbow-down configurations

This approach was preferred over numerical methods due to its computational efficiency, analytical clarity, and suitability for real-time applications.

3.3 Dynamic modelling approach

The dynamic behaviour of the manipulator was considered using the standard rigid-body robot dynamics formulation [3]:

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q)$$

where τ is the vector of joint torques, $M(q)$ is the inertia matrix, $C(q, \dot{q})$ represents Coriolis and centrifugal effects, and $G(q)$ is gravity.

This expression provides the dynamic model that is incorporated in simulation and modelling used to extend the kinematic model developed in MATLAB. However, a complete rigid-body dynamic parameterisation was not fitted to the lack of detailed physical parameters such as link masses, mass centres, and inertia tensors from the manufacturer. Accordingly, the dynamic analysis was focused on establishing the dynamic form of the equations and explaining how such terms would be included in a MATLAB/Simulink model for future extension.

3.4 MATLAB Implementation

The robot model was implemented in MATLAB using a modular approach. Separate MATLAB functions were created for the Denavit-Hartenberg transformation, forward kinematics evaluation, inverse kinematics solution, Jacobian computation, singularity analysis, and validation. This structure allows each component to be tested independently and reused in later analysis and simulation tasks, which were then evaluated numerically during testing [1].

The implementation was designed so that a single configuration input could be provided as input, the corresponding joint angles could be computed through inverse kinematics, and the resulting configuration could then be verified through forward kinematics reconstruction. Additional checks were included to detect unreachable targets and singular configurations.

3.5 Testing Framework

A structured testing framework was used to validate the correctness of the mathematical model and the MATLAB implementation. The forward kinematics was tested by providing known joint configurations and comparing the resulting end-effector pose with expected values. The inverse kinematics was validated by calculating joint angles for joint combinations from forward kinematics and substituting the solution back into the forward kinematics equations. Additional tests were used to verify consistency between both solutions for reachable targets. Together, these checks ensured internal consistency between the analytical model, the implementation and the computational implementation.

4 Results and Validation

The MATLAB model was validated using forward-kinematics, inverse-kinematics and singularity test cases. Forward kinematics was checked by comparing computed end-effector poses with expected values from known joint configurations. Inverse kinematics was validated by reconstructing the end-effector pose from the target Cartesian position. Additional tests were used to confirm singularity detection and the generation of unreachable targets.

Table 2: Validation of forward and inverse kinematics test results (table caption)

Test case	Input	Output
Forward kinematics	$q = (0.5236, 0.7854, 0, 0)$ rad	$(x, y, z) = (0.1847, 0.106, 0.5)$
Inverse kinematics	$(x, y, z) = (0.2, 0.1, 0.5)$	$q = (0.5236, 0.7854, 0, 0)$
Singularity test	$\theta_3 = 0$	$\det(J) = 0$
Unreachable target	Multiple valid; invalid cases	[1], [1] omitted

The validation results show good agreement between the analytical model and the MATLAB implementation. In particular, the consistency between the forward-kinematics and inverse-kinematics results confirms that the implemented solution for a position target is internally correct. The singularity and unreachable target tests further demonstrate that the implementation correctly handles important edge cases within the model's kinematic limits.

5 Discussion and Conclusion

This project developed and validated a kinematic model of the Dobot Magician Lite using the Denavit–Hartenberg formulation, forward kinematics, inverse kinematics in MATLAB. Forward and inverse kinematics were derived analytically, the Jacobian was used to study velocity mapping and singularities, and simulation figures were identified from the analytical model. The resulting MATLAB implementation was used to reproduce valid end-effector poses and report targets lying outside the reachable workspace.

The passive parallelogram mechanism was represented by the physical parallelogram mechanism of the robot within a reduced kinematic model. This choice was driven by replacing the closed-chain behaviour with an equivalent open-chain model containing a passive transformation, which preserves the geometric constraint without introducing an additional actuated degree of freedom. Accordingly, the kinematic model provides a robust analytical and computational framework for studying the motion of the Dobot Magician Lite. The model is suitable for forward kinematics, inverse kinematics, inverse kinematics, singularity detection and validation testing in MATLAB. Future work could extend the model to include more dynamically detailed dynamic parameters, actuator constraints, joint limits and closed-loop control in order to improve realism and practical applicability.

Figure 6: Successful inverse kinematics solution for the target (0.2, 0.1, 0.5).

Figure 7: Detection of unreachable target in inverse kinematics.

The system correctly identifies when a target position lies outside the robot's reachable workspace. In this example, the computed distance 4 exceeds the maximum reach of the robot, $L_2 + L_3 = 0.30$ m, confirming that no solution exists. This demonstrates that the inverse kinematics implementation includes proper workspace validation and prevents physically invalid solutions.

6 References

References

[1] P. Corke. *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*. Springer, Berlin, 2nd edition, 2017.

[2] J. J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson Education, Upper Saddle River, 3rd edition, 2005.

[3] J. Denavit and R. S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22:215–221, 1955.

[4] Dobot Magician Lite Robot Arm Kit. *DobotArm-kit*, n.d. [Online]. Available: https://download.dobot.cc/product-manual/magician-lite/DobotArm-kit/en/01Introduction/20Introduction.html, n.d.

[5] MathWorks. Import and apply inverse kinematics to robot arm, 2023. [Online]. Available: https://uk.mathworks.com/help/ros/ug/import-and-apply-inverse-kinematics-to-robot-arm.html.

[6] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. *Robotics: Modelling, Planning and Control*. Springer London, 2009.

[7] M. W. Spong, S. Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, Hoboken, 2006.